

Lineamientos de Infraestructura Cloud con Terraform

Realizado Por	Fecha	Firma
Royss Hugo Martel Massi	18/03/2025	
Revisado Por	Fecha	Firma
Jonathan Franchesco Torres Baca	10/05/2025	
Aprobado Por	Fecha	Firma
Jose Jesus Salazar Silva	11/05/2025	

Objetivo [🔗](#)

Este documento establece los lineamientos para implementar y gestionar infraestructura como código (IaC) con Terraform, garantizando escalabilidad, automatización y consistencia en entornos cloud. Además, busca optimizar costos y reducir errores manuales en la provisión de recursos.

Alcance [🔗](#)

Estos lineamientos aplican a entornos AWS, GCP y Azure, considerando su implementación en proyectos de diferentes tamaños. Se enfocan en modularización, seguridad, nomenclatura y buenas prácticas para asegurar una administración eficiente de los recursos cloud.

Organización de Archivos [🔗](#)

Para mantener un código ordenado y reutilizable, se debe respetar la siguiente estructura de carpetas:

```
1 /terraform
2 |─ /modules
3 | |─ /vpc
4 | | |─ local.tf
5 | | |─ main.tf
6 | | |─ variables.tf
7 | | |─ outputs.tf
8 | |─ /ec2
9 | | |─ main.tf
10 | | |─ variables.tf
11 | | |─ outputs.tf
12 |─ main.tf
13 |─ outputs.tf
14 |─ data.tf
15 |─ locals.tf
16 |─ providers.tf
17 |─ variables.tf
18 |─ versions.tf
19 |─ backend.tf
20 |─ terraform.tfvars
```

Se recomienda excluir los siguientes archivos en `.gitignore` para evitar exponer datos sensibles o innecesarios:

✓ Ejemplo `.gitignore`

```
1 *.tfstate
2 *.tfvars
3 .terraform/
4 .terraform.lock.hcl
```

Lineamientos Generales [🔗](#)

- Agrupar recursos relacionados en módulos reutilizables.
- No hardcodear datos sensibles en el código.
- Revisar siempre los cambios con `terraform plan` antes de aplicarlos.
- Etiquetar recursos para facilitar el seguimiento de costos.
- Evitar modificar recursos aprovisionados con Terraform desde la consola del proveedor de nube.
- Usar `terraform import` para gestionar recursos en la nube existentes en `terraform.tfstate`.
- Usar Terraform Registry para el uso e implementación de providers.
- Guardar en un repositorio de Azure DevOps el código fuente de la solución de Terraform.
- Documentar los módulos con `README.md` que incluyan ejemplos de uso y descripción de variables.
- Utilizar meta-argumentos `count` y `for_each` para crear múltiples recursos similares.
- Definir `outputs.tf` para exponer información relevante para integración con otros módulos.
- Implementar límites de versión para los providers (`>=` y `~>`) para evitar actualizaciones disruptivas.
- Implementar lifecycle blocks para proteger recursos críticos contra destrucción accidental.
- Utilizar data sources para consumir información de recursos existentes.
- No ejecutar el comando `destroy` en el ambiente productivo, corregir la solución con `apply` o si es algo en específico con `apply -target`.

Lineamientos de Seguridad [🔗](#)

- Utilizar gestores de secretos (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault).
- Habilitar locking de estado remoto (ej. `DynamoDB` en AWS, `GCS` en GCP).
- No declarar valores por defecto en `variables.tf`, usar `terraform.tfvars`.
- Evitar exponer variables en `terraform.tfvars`.
- Agregar `.gitignore` para asegurar que archivos como `.terraform/`, `*.tfstate`, y `*.tfvars` no se suban al repositorio.
- Los roles/usuarios que ejecutan Terraform en local o en pipelines deben tener permisos estrictamente necesarios.
- Utilizar almacenamiento remoto (`backend.tf`) con versionamiento y cifrado.
- No permitir `resource "*"` , ni permisos globales en políticas.
- Asegurar de que no haya dependencia cruzada no deseada entre entornos.

Lineamientos de Nomenclatura [🔗](#)

Para mantener un código claro, escalable y fácil de mantener, es fundamental seguir buenas prácticas en la nomenclatura de recursos, variables y módulos.

Nombre de recursos [🔗](#)

Reglas: [🔗](#)

1. **Evitar nombres genéricos** como `my_instance` o `test_vpc`.
2. **Usar snake_case** (guiones bajos) para mayor legibilidad.
3. **No incluir valores cambiantes** (Ejemplo: evitar `instance_01`, mejor `web_server`).
4. **Usar nombres descriptivos** `<proveedor>_<tipo>_<propósito>` (ej. `aws_s3_bucket_logs`)

✅ Ejemplo `main.tf`:

```
1 resource "google_cloudfunctions2_function" "gcp_cloud_run_functions" {
2   # Configuración del recurso
3 }
```

⚠️ Es crucial destacar que el nombramiento de recursos forma parte del desarrollo de la infraestructura como código (IaC) y no guarda relación con el nombre del servicio aprovisionado en la nube. Cada servicio debe seguir su propio **lineamiento de nomenclatura establecido**.

aws : [📄 Lineamiento de Nomenclatura de Servicios AWS](#)

gcp : Aun por definir

azure : Aun por definir

Nombres de Variables [🔗](#)

Reglas: [🔗](#)

1. **Usar snake_case** para mayor claridad (`db_username`, `vpc_id`).
2. **Nombrar según el propósito** y evitar abreviaciones innecesarias.
3. **Añadir prefijos según el contexto** (`aws_`, `gcp_`, `az_` si es multi-cloud).
4. **No incluir valores específicos** en el nombre.

✅ Ejemplo `variables.tf`:

```
1 variable "bucket_name" {
2   description = "Nombre del bucket donde se almacenara los archivos csv"
3   type       = string
4 }
```

Nombres de Módulos [🔗](#)

Reglas: [🔗](#)

1. **Usar nombres en plural** si representan un conjunto de recursos (`networking`, `databases`).
2. **Evitar nombres genéricos** (`module1`, `custom_module`).
3. **Prefijar según su función** (`vpc_`, `bucket_`, `ec2`, `lambda_`)
4. **Usar nombres coherentes** con los directorios donde están almacenados.

✅ Ejemplo `main.tf`:

```
1 module "bucket_csv" {
2   source      = "../modules/bucket"
3   bucket_name = var.bucket_name
4   region     = var.region
5   functions_name = var.functions_name
```

```

6 }
7
8 module "fun_get_session_calimaco" {
9     source          = "./modules/cloud_run_function"
10    # Variables de entrada
11 }
12
13 module "fun_close_session_calimaco" {
14     source          = "./modules/cloud_run_function"
15    # Variables de entrada
16 }

```

Importante: Los módulos deben ser reutilizables y seguir buenas prácticas de organización. Si por restricciones de configuración no es posible hacer un módulo reutilizable, es recomendable separarlo en carpetas con nombres específicos para mantener claridad y orden.

Nombres de Tags [🔗](#)

Reglas: [🔗](#)

1. **Usar snake_case** para mayor claridad (name, iac_version).
2. **Nombrar según el propósito** y evitar abreviaciones innecesarias.
3. **No incluir valores específicos** en el nombre.
4. **Respetar las etiquetas obligatorias** en el apartado [Etiquetado de Recursos](#).

✅ Ejemplo `terraform.tfvars`:

```

1 tags = {
2     "environment"      = "dev"
3     "vertical_business" = "caf"
4 }

```

Buenas Prácticas [🔗](#)

Este apartado establece recomendaciones clave para el proyecto, enfocadas en mejorar la legibilidad, facilitar el mantenimiento y asegurar la coherencia en el código.

Uso de Versiones Específicas [🔗](#)

Para evitar incompatibilidades, se debe especificar versiones y proveedores

✅ Ejemplo `versions.tf`

```

1 terraform {
2     required_version = "~>1.11.0"
3     required_providers {
4         google = {
5             source = "hashicorp/google"
6             version = "~>6.24.0"
7         }
8     }
9 }

```

Testing y Validación [🔗](#)

Se recomienda usar las siguientes herramientas:

- `tflint` : Analiza errores en código.
- `tfsec` : Escanea vulnerabilidades en configuraciones.
- `infracost` : Estima costos en la nube.
- `tfenv` : Gestionar versiones de.

Separación de variables [↗](#)

Se deben definir nombres de variables claros, acordes al dato que almacenarán, especificando el tipo de dato, sensibilidad y su descripción en `variables.tf`. Los valores correspondientes deben proporcionarse en `terraform.tfvars`.

✓ Ejemplo `variables.tf`.

```
1 variable "project_id" {
2   description = "ID del proyecto GCP"
3   type       = string
4   sensitive  = true
5 }
6
7 variable "functions_name" {
8   description = "Nombre de las funciones"
9   type       = list(string)
10 }
```

✓ Ejemplo `terraform.tfvars`.

```
1 project_id      = "dev-acity-sandbox"
2 functions_name = ["get-session-calimaco", "close-session-calimaco"]
```

Etiquetado de Recursos [↗](#)

Se deben definir etiquetas obligatorias para mejorar la gestión, monitoreo, trazabilidad y optimización de costos de los recursos en la nube.

Predeterminadas Obligatorias : [↗](#)

Etiquetado	Descripción	Ejemplo
<code>environment</code>	Entorno en el que se usa el recurso.	dev
<code>project</code>	Nombre del proyecto	cabono-automatization
<code>owner</code>	Célula, área o persona encargada del recurso	rmartel@acity.com.pe
<code>vertical_business</code>	Identifica la vertical de negocio	caf
<code>iac</code>	Indica la herramienta de gestión.	Terraform
<code>iac_version</code>	Indica la versión de la herramienta de gestión.	~>1.12.0
<code>region</code>	Región donde se va implementar la infraestructura	us-central1

Obligatorias por Recurso: [↗](#)

Etiquetado	Descripción	Ejemplo
------------	-------------	---------

name	Nombre del servicio que se va aprovisionar	lambda_col_calimaco_process_order
------	--	-----------------------------------

Configuración de Etiquetas Predeterminadas: [🔗](#)

✓ Ejemplo variables.tf

```
1 variable "tags" {
2     description = "tags"
3     type        = map(string)
4 }
```

✓ Ejemplo terraform.tfvars

```
1 tags = {
2     "environment"    = "dev"
3     "project"        = "cabono-automatization"
4     "owner"          = "rmartel@acity.com.pe"
5     "vertical_business" = "caf"
6     "iac"            = "terraform"
7     "iac_version"    = "~>1.12.0"
8     "region"         = "us-central1"
9 }
```

✓ Ejemplo providers.tf

```
1 provider "aws" {
2     region = "us-east-1"
3     default_tags {
4         tags = var.tags
5     }
6 }
```

Configuración de sufijo y Etiqueta por Recurso: [🔗](#)

✓ Ejemplo local.tf

```
1 locals {
2     sufix = "${var.tags.vertical_business}-${var.tags.project}"
3 }
```

✓ Ejemplo main.tf

```
1 module "vpc" {
2     source      = "./modules/vpc"
3     vpc_name    = "vpc-${local.sufix}-public"
4 }
```

✓ Ejemplo vpc / variables.tf

```
1 variable "vpc_name" {
2     description = "Nombre de la vpc pública"
3     type        = string
4 }
```

✓ Ejemplo vpc/main.tf

```
1 resource "aws_vpc" "aws_vpc_public" {
2     name = var.vpc_name
```

```

3 tags = {
4   Name = var.vpc_name
5 }
6 }

```

Respaldo del archivo terraform.tfstate [🔗](#)

Debe almacenarse en un bucket remoto con versionamiento habilitado. Esto previene la pérdida de datos, protege la confidencialidad y garantiza la integridad.

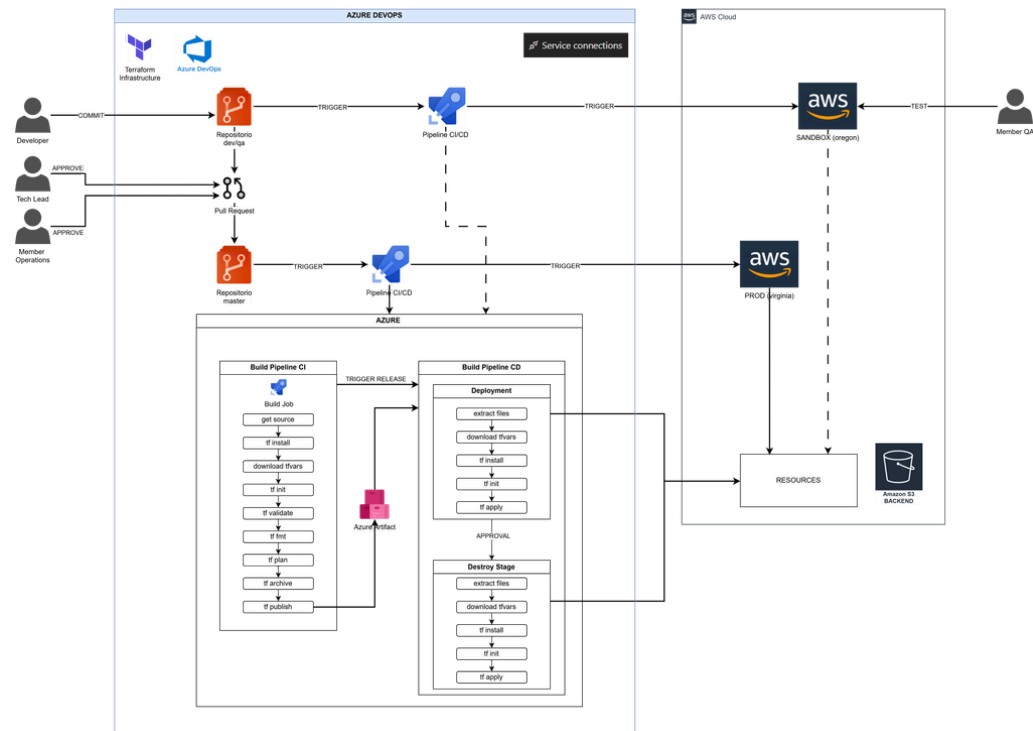
✅ Ejemplo backend.tf

```

1 terraform {
2   backend "gcs" {
3     bucket = "terraform-backends"
4     prefix = "cabono-automatization/state"
5   }
6 }

```

Flujo de Trabajo [🔗](#)



Matriz RACI [🔗](#)

Tareas e Interesados	Desarrollador	QA	Líder Técnico	Operaciones
Diseñar y estructurar el código de infraestructura.	R		A	
Supervisar la ejecución del pipeline CI/CD en el entorno sandbox.	A	R	I	C

Versionar la infraestructura.	R	I	A	
Validar entornos desplegados automáticamente.	A	R	I	C
Ejecutar pruebas funcionales sobre la infraestructura provisionada.		R	A	C
Asegurar la trazabilidad de los cambios.		R	A	I
Guiar técnicamente al equipo de desarrollo.			R	C
Revisar y aprobar PRs de la infraestructura.	I		R/A	R/A
Implementar y ejecutar despliegues controlados en ambientes.	I	I	A	R
Monitorear el estado de los recursos en la nube.		A	I	R

Conclusión [🔗](#)

Este documento proporciona una guía para la implementación eficiente de infraestructura con **Terraform**, promoviendo modularización, seguridad y buenas prácticas. Es importante mantener estos lineamientos actualizados y aplicar revisiones periódicas para mejorar la calidad del código y la gestión de recursos en la nube.